

SESSION 2 / 4

From LLM to agent: building an agent

ReAct, tool calling, memory — and the first autonomous agents

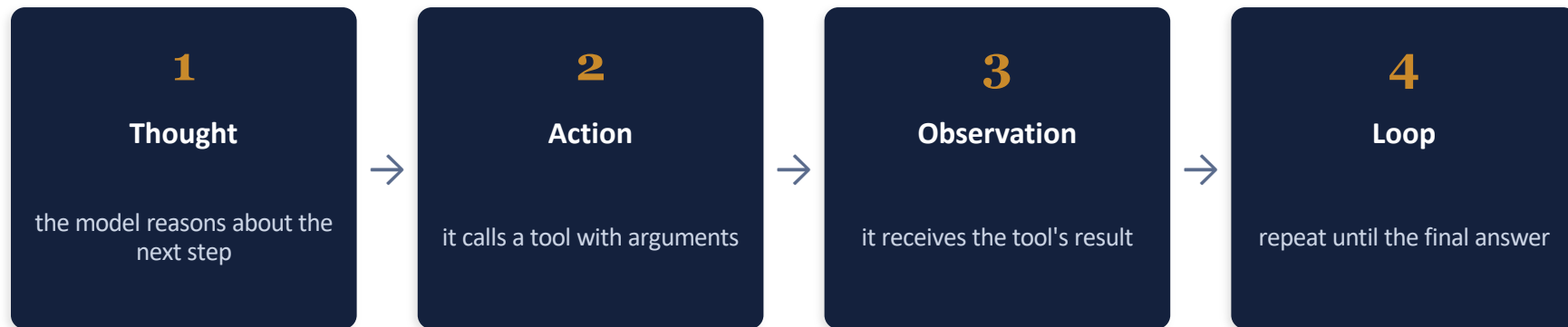
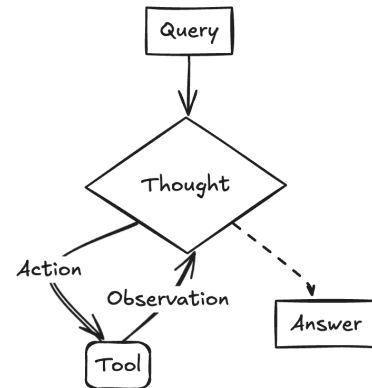
Guillaume Hochard · Airpanel | 22–24 June 2026

Why LLMs make agency possible

- Natural-language reasoning: break a problem down, plan steps.
- Generalization in-context: a new task without retraining, just via the prompt.
- Tool calling: the model produces a structured call (name + JSON arguments).
- It is the missing link: it can now act, not just describe.
- Limit: without tools or a loop, the LLM remains a 'one-shot' generator that verifies nothing.

The ReAct paradigm

Reasoning + Acting (Yao et al., 2022): alternating thought, action and observation.



Anatomy of a tool call

The model returns not text but a structured call that YOUR code executes.

1) We describe the tool (JSON schema)

```
tool = {"type": "function", "function": {  
  "name": "get_current_time",  
  "description": "Returns the current time",  
  "parameters": {"type": "object", "properties": {}}}}
```

2) The model DECIDES to call the tool

```
reply.tool_calls -> [{"name": "get_current_time", "arguments": {}}]
```

3) Our code executes, then returns the observation to the model

```
messages.append({"role": "tool", "content": "2026-06-22 09:15"})
```

Anatomy of an LLM agent

Planner

decides what to do next, breaks down the task.

E.g. "first search, then compute"

Tools

callable functions: search, computation, API, code.

E.g. calculator, web_search

Memory

short-term context + long-term knowledge (RAG).

E.g. history + vector store

Executor

the loop that orchestrates calls and observations.

E.g. run_agent() from Lab 2

An agent's memory

Context is limited and costly: you must decide what to keep.

- Working memory: the conversation history present in the context.
- Long-term memory: facts persisted then retrieved as needed (RAG — session 3).
- Episodic memory: traces of past runs to improve / self-correct.
- Trade-off: putting everything in the prompt is expensive and dilutes attention.

AutoGPT & BabyAGI: promises vs limits

Promises (2023)

- 100% autonomous agents that self-decompose into sub-tasks
- Goal → plan → execution loop without intervention
- First mainstream demonstration of self-orchestration

Observed limits

- Loops, drift and runaway costs
- Fragility on long / ill-defined tasks
- Unreliable in production without guardrails or supervision

These frameworks were popular two years ago but are now outdated
Frameworks mostly used in the industry are langchain, agno, crew ai, open ai sdk ...

Recurring pitfalls to anticipate

- Tool hallucination: calling a non-existent tool or making up arguments.
- Loops: the same tool called again with no progress → set a step limit.
- Cost & latency that grow with the number of steps.
- Sensitivity to external content (prompt injection — covered in session 4).

Making the agent reliable: guardrails & self-critique

From fragile prototype to robust agent — what you add in Lab 2.

Anti-loop

reject an identical tool call already made

Allow-list

only allow tools that are explicitly intended

“I don't know”

require admitting ignorance if there is no source

Self-critique (Reflexion)

re-read its answer, then correct if needed



Lab 2 — ReAct agent coded by hand

Notebook: `lab_02_agent_react_from_scratch.ipynb`

- Implement the ReAct loop without a framework (call → tool calls → observations).
- Equip the agent with several tools: calculator, local search, date.
- Trigger and observe a tool error, a loop and a hallucination.
- Add a first guardrail (anti-loop, 'I don't know').

Key takeaways



- An LLM agent = planner + tools + memory + execution loop.
- ReAct alternates reasoning, action and observation.
- Function calling connects the model to the world — but it is YOUR code that executes.
- AutoGPT/BabyAGI: inspiring, but fragile without control (and outdated now).
- Next step: industrialize this loop with frameworks.